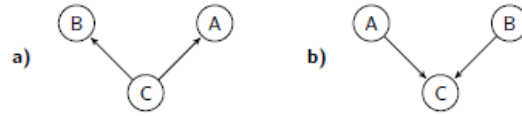


Homework 1: Bayesian Networks and Gene Regulation

Bernhard Buss, Timothy Lauren Nijhuis, Carlo Rossetto and Gökem Solun

Problem 1: Conditional independence and Bayesian networks

Consider the following graphical structures, corresponding to (different) Bayesian networks. For which network does the statement $A \perp B \mid C$ hold? For which does the statement $A \perp B$ hold? Prove your answers by the laws of probability.



1a) $A \perp B \mid C$ holds for Network a, $A \perp B$ does not.

Network a is a common cause structure/fork. Here, the node C is a parent to both A and B. The arrows go from C to A and C to B. In this structure, information flows from C to the children. If the state of C is known and conditioned upon, A and B become independent of each other because the common cause explains both.

$$P(A, B, C) = P(A|C)P(B|C)P(C)$$

which is substituted in the following

$$P(A, B|C) = \frac{P(A, B, C)}{P(C)} = \frac{P(A|C)P(B|C)P(C)}{P(C)} = P(A|C)P(B|C) \Rightarrow A \perp B \mid C$$

However, since C is not null ($\emptyset$), they are not marginally independent, meaning $A \not\perp B$.

Alternatively, the fact they are not marginally independent can also be shown by:

$$P(A, B) = \sum_C P(A, B, C) = \sum_C P(A|C)P(B|C)P(C) = \sum_C \frac{P(C|B)P(B)}{P(C)} \frac{P(C|A)P(A)}{P(C)} P(C)$$

$$= P(A)P(B) \sum_C \frac{P(C|B)P(C|A)}{P(C)} \neq P(A)P(B) \Rightarrow A \not\perp B$$

1b) $A \perp B \mid C$ does not hold, for network B. $A \perp B$ does.

Network b is a common effect structure/collider. Here, A and B are both parents of C. Arrows point from A and B into C. In a collider, the parents are marginally independent. They only become dependent if you condition on the child C or one of its descendants. Since we are not conditioning on C here, A and B remain independent.

$$P(A, B, C) = P(A)P(B)P(C|A, B)$$

which is substituted in the following

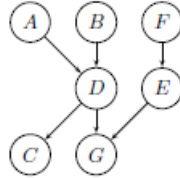
$$P(A, B|C) = \frac{P(A, B, C)}{P(C)} = \frac{P(A)P(B)P(C|A, B)}{P(C)} \neq P(A|C)P(B|C) \Rightarrow A \not\perp B \mid C$$

While

$$\begin{aligned} P(A, B) &= \sum_C P(A, B, C) = \sum_C P(A)P(B)P(C|A, B) = P(A)P(B) \sum_C P(C|A, B) \\ &= P(A)P(B) \cdot 1 = P(A)P(B) \Rightarrow A \perp B \end{aligned}$$

Problem 2: Markov blanket

Consider the following graphical structure of a Bayesian network:



Determine the Markov blanket $MB(D)$ of a node D and show that the conditional probability $P(D|A, B, C, E, F, G)$ is the same as $P(D|MB(D))$.

2) The Markov Blanket (MB) of a node in a Bayesian Network consists of three specific groups of neighbors: parents, children, and other parents of its children.

Parents of D: A, B. Children of D: C, G. Spouses of D: Child C: D and Child G: D, E.

$MB(D) = A, B, C, E, G$.

$$P(D|A, B, C, E, G) = P(D|MB(D))$$

$$\begin{aligned} P(D|A, B, C, E, F, G) &= \frac{P(A, B, C, D, E, F, G)}{P(A, B, C, E, F, G)} = \frac{P(A, B, C, D, E, F, G)}{\sum_D P(A, B, C, D, E, F, G)} = \\ &= \frac{P(A)P(B)P(C|D)P(D|A, B)P(F)P(E|F)P(G|D, E)}{\sum_D P(A)P(B)P(C|D)P(D|A, B)P(F)P(E|F)P(G|D, E)} \end{aligned}$$

Since the bottom sum is over d , terms not containing d can be factored out.

$$= \frac{P(A)P(B)P(F)P(E|F)P(C|D)P(D|A, B)P(G|D, E)}{P(A)P(B)P(F)P(E|F) \sum_D P(C|D)P(D|A, B)P(G|D, E)} = \frac{P(C|D)P(D|A, B)P(G|D, E)}{\sum_D P(C|D)P(D|A, B)P(G|D, E)}$$

the terms left (A, B, C, E and G) are the members of the Markov blanket of D , there is no F . D is conditionally independent of F , given the Markov blanket. So...

$$P(D|A, B, C, E, F, G) = P(D|A, B, C, E, G) = P(D|MB(D))$$

Problem 3: Learning Bayesian networks from protein data

In this exercise, we will use the R package BiDAG to learn Bayesian networks from protein data. Download the data [here](#). The data consists of the measurements of 11 phosphorylated proteins and phospholipids derived from primary immune system cells, subjected to both general and specific molecular interventions. (Hint: read the help files of the package and use default parameters unless otherwise stated.)

- (a) First, run `set.seed(42)` for reproducibility. Read in the data in R. Report the number of variables n and the number of observations N . Visualize the data in a pair plot using the `ggpairs` function in the R package `GGally`. Randomly split the data into 80% training data and 20% test data. Initialize the parameters using the function `BiDAG::scoreparameters` with the training data and the Bayesian Gaussian equivalent (BGe) score. [Note: The BGe score is a fully-decomposable marginal likelihood function $P(D | G)$ for scoring Bayesian networks. The main underlying assumption is that the data is normally distributed with $N(\mu, W^{-1})$. The precision matrix W follows a Wishart prior $W \sim n(T^{-1}, a_w)$, where $a_w > n - 1$ is the degrees of freedom and T is the positive definite parametric matrix. The mean vector μ follows a normal prior $N(v, W a_\mu)$ with $a_\mu > 0$.]

Run `install.packages("BiDAG")` in the R console. Then load the package by running `library(BiDAG)`. You may need to install the missing packages `graph`, `RBGL`, and `Rgraphviz` from Bioconductor.

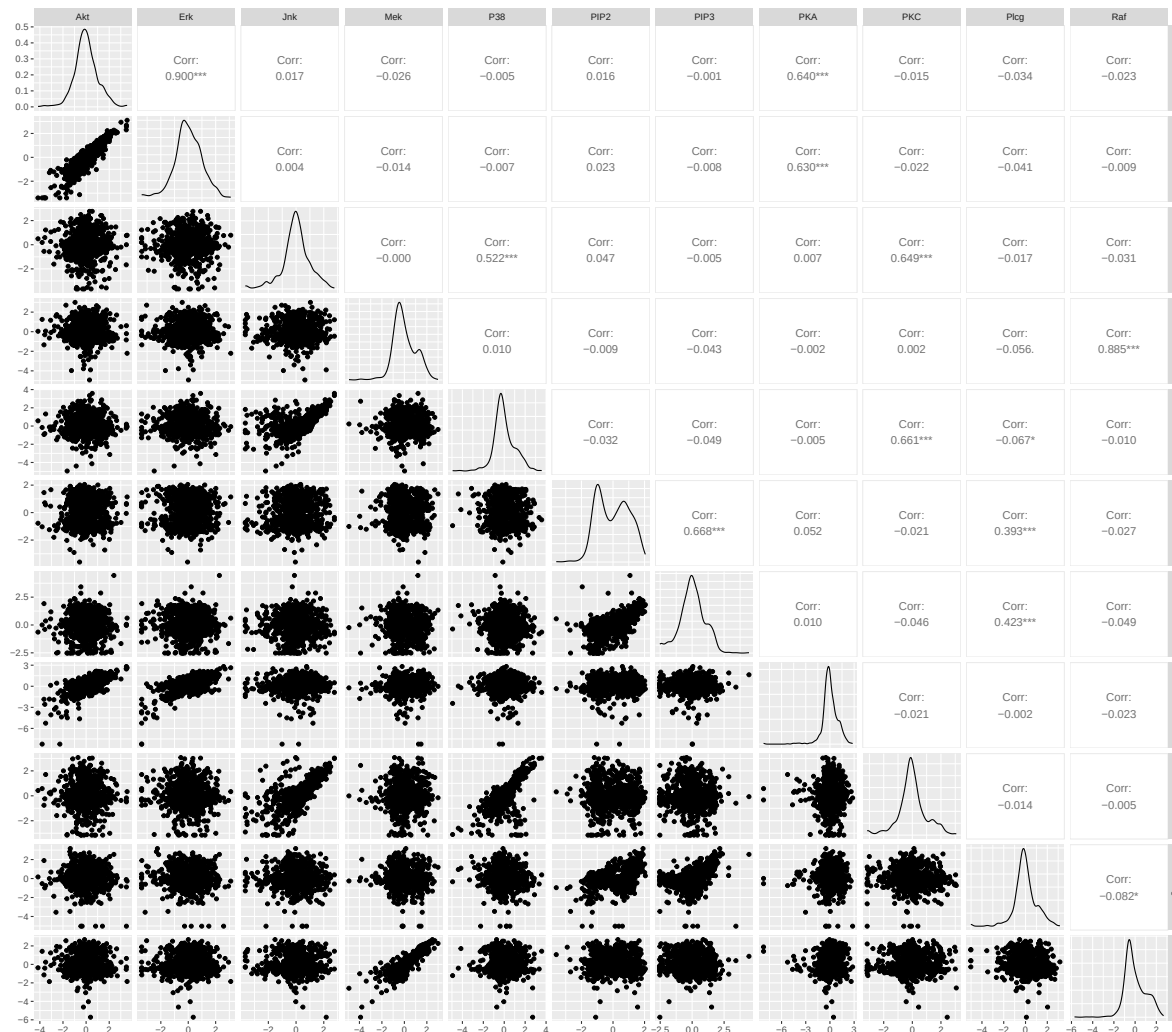
```
set.seed(42)
data <- read.csv("2005_sachs_2_cd3cd28icam2_log_std.csv")
n <- ncol(data)
N <- nrow(data)
cat("Number of variables (n):", n, "\n")
```

Number of variables (n): 11

```
cat("Number of observations (N):", N, "\n")
```

Number of observations (N): 902

```
suppressPackageStartupMessages(library(GGally))
ggpairs(data)
```



```
train_indices <- sample(seq_len(N), size = floor(0.8 * N))
train_data <- data[train_indices, ]
test_data <- data[-train_indices, ]

suppressPackageStartupMessages(library(BiDAG))
myScore <- scoreparameters(scoretype = "bge", data = train_data)
print(myScore)
```

object of class 'scoreparameters'

\$type
bge

```
$data
data contains 721 observations of 11 variables
```

```
$DBN
FALSE
```

```
$am
1
```

```
$aw
1
```

```
$means
-0.01224145 ... 0.02121122
```

```
$SigmaN
matrix 11 x 11
```

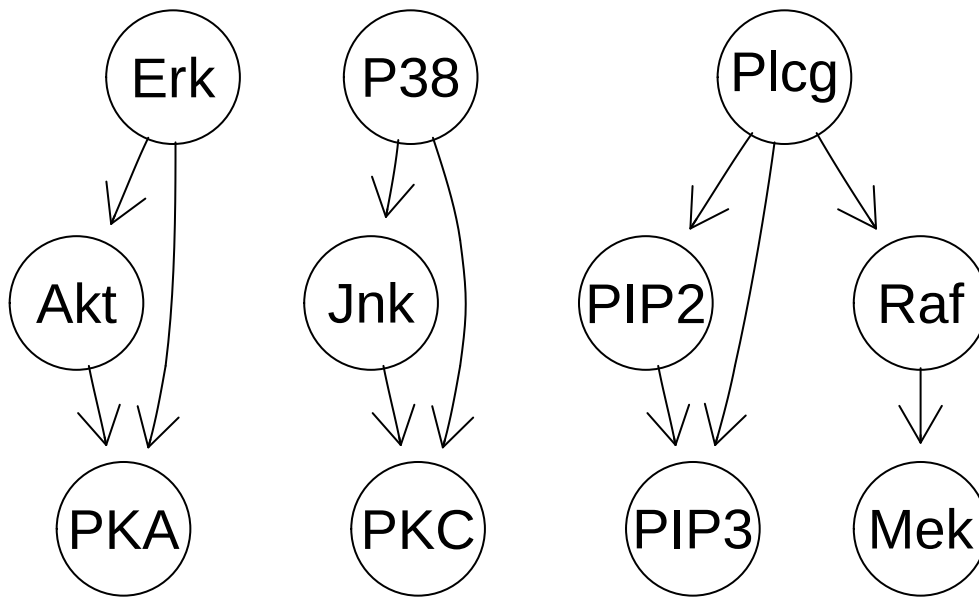
- (b) Learn a Bayesian network using the `BiDAG::iterativeMCMC` function. Plot the directed acyclic graph (DAG). Evaluate the log BGe score of the test data against the estimated DAG using the function `BiDAG::scoreagainstDAG`.

```
suppressPackageStartupMessages(library(BiDAG))
suppressPackageStartupMessages(library(Rgraphviz))
suppressPackageStartupMessages(library(graph))

myDAG <- iterativeMCMC(myScore)
```

```
maximum parent set size is 3
core space defined, score table are being computed
score tables completed, iterative MCMC is running
```

```
g <- as(myDAG$DAG, "graphNEL")
plot(g)
```



```

testScore <- scoreparameters(scoretype = "bge", data = test_data)

test_log_score <- sum(scoreagainstDAG(scorepar = testScore, incidence = myDAG$DAG))

cat("Log BGe score of test data against estimated DAG:", test_log_score, "\n")

```

Log BGe score of test data against estimated DAG: -2195.537

A more negative (lower) score might indicate that the model is overfitted, because the patterns learned from the training data do not generalise well to the new, unseen test data.

- (c) Assume you receive new data where only phosphorylated proteins are measured but the phospholipids, PIP2 and PIP3, are not. You would like to impute PIP2 and PIP3 from (a subset of) the other variables. According to the Bayesian Network you learned in (b), which subset should you choose and why?

One can observe that $MB(PIP2) = \{PIP3, Plcg\}$ and $MB(PIP3) = \{PIP2, Plcg\}$. Since both PIP2 and PIP3 are unknown, one can only rely on Plcg.

So, if only using observed variables: PIP3 from $\{Plcg\}$, PIP2 from $\{Plcg\}$.

If sequential imputation: first impute PIP2 from $\{Plcg\}$, then impute PIP3 from $\{Plcg, PIP2\}$.